

Trabalho Prático II – Aprendizado de Máquina

Introdução à Inteligência Artificial

Mateus Rabelo de Souza Melo

Felipe Almeida Pereira

Junho de 2026

Resumo

Este relatório apresenta a implementação e a avaliação de dois algoritmos clássicos de aprendizado de máquina aplicados ao conjunto de dados *NBA Rookie Stats*: o *k-Nearest Neighbors* (kNN), utilizado em uma tarefa supervisionada de classificação, e o k-Means, utilizado em uma tarefa não-supervisionada de agrupamento. O objetivo do kNN é prever se a carreira de um jogador novato da NBA durará pelo menos cinco anos, enquanto o objetivo do k-Means é investigar se os atributos estatísticos dos jogadores formam grupos coerentes com o rótulo real `TARGET_5Yrs`. Além da implementação própria dos algoritmos, os resultados foram comparados com implementações equivalentes da biblioteca *scikit-learn*. Os experimentos indicaram que o kNN obteve melhor desempenho para valores maiores de k , com destaque para $k = 50$ na implementação própria e $k = 15$ no *scikit-learn*. Já o k-Means não separou perfeitamente as classes originais, mas identificou grupos com forte relação com volume de jogo, minutos em quadra e permanência na liga.

Sumário

1	Introdução e Objetivo	3
2	Metodologia	3
2.1	Base de dados	3
2.2	Normalização dos dados	4
2.3	k-Nearest Neighbors	4
2.4	k-Means	5

2.5	Métricas de avaliação	5
3	Experimentos e Resultados	6
3.1	Resultados do kNN	6
3.2	Comparação do kNN com o scikit-learn	7
3.3	Resultados do k-Means para $k = 2$	8
3.4	Resultados do k-Means para $k = 3$	9
3.5	Comparação do k-Means com o scikit-learn	10
4	Discussão	11
5	Conclusão	12
	Referências	13

1 Introdução e Objetivo

O aprendizado de máquina é uma área da Inteligência Artificial que busca construir modelos capazes de identificar padrões em dados e utilizá-los para tomar decisões, realizar previsões ou organizar informações. Neste trabalho, foram estudados dois tipos de aprendizado: o aprendizado supervisionado, no qual os dados de treinamento possuem rótulos conhecidos, e o aprendizado não-supervisionado, no qual o algoritmo deve encontrar estruturas nos dados sem utilizar rótulos durante o treinamento.

O conjunto de dados utilizado foi o *NBA Rookie Stats*, composto por estatísticas de jogadores novatos da NBA. Cada amostra representa um jogador e contém atributos relacionados ao seu desempenho, como jogos disputados, minutos jogados, pontos por jogo, arremessos, rebotes, assistências, roubos de bola, bloqueios e perdas de bola. A variável de interesse é `TARGET_5Yrs`, que indica se a carreira do jogador durou pelo menos cinco anos na liga, assumindo valor 1 para carreiras de cinco anos ou mais e valor 0 caso contrário.

O objetivo geral do trabalho foi implementar e analisar os algoritmos kNN e k-Means. No caso do kNN, o objetivo específico foi classificar os jogadores do conjunto de teste de acordo com a permanência ou não por pelo menos cinco anos na NBA. No caso do k-Means, o objetivo foi agrupar os jogadores usando apenas os atributos de entrada, sem considerar o rótulo `TARGET_5Yrs`, e posteriormente verificar se os agrupamentos encontrados apresentam relação com as classes reais.

Também foi realizada uma comparação com a biblioteca *scikit-learn*, a fim de avaliar se as implementações desenvolvidas produziram resultados coerentes com uma biblioteca amplamente utilizada em aprendizado de máquina.

2 Metodologia

2.1 Base de dados

Foram utilizados dois arquivos: `nba_treino.csv` e `nba_teste.csv`. O conjunto de treino contém 1.072 observações, correspondendo a 80% da base, enquanto o conjunto de teste contém 268 observações, correspondendo aos 20% restantes. Ao todo, a base possui 1.340 observações.

Cada jogador é representado por 19 atributos preditivos numéricos e por uma variável resposta binária. Os atributos preditivos são:

Atributo	Descrição	Atributo	Descrição
GP	Jogos disputados	FT%	Aproveitamento em lances livres
MIN	Minutos jogados	OREB	Rebotes ofensivos
PTS	Pontos por jogo	DREB	Rebotes defensivos
FGM	Arremessos convertidos	REB	Rebotes totais
FGA	Tentativas de arremesso	AST	Assistências
FG%	Aproveitamento de arremessos	STL	Roubos de bola
3PMade	Cestas de 3 convertidas	BLK	Bloqueios
3PA	Tentativas de 3 pontos	TOV	Perdas de bola
3P%	Aproveitamento de 3 pontos	TARGET_5Yrs	Rótulo da classe
FTM	Lances livres convertidos	FTA	Tentativas de lances livres

A distribuição da variável resposta no conjunto de teste é composta por 100 jogadores da classe 0 e 168 jogadores da classe 1. Assim, há uma maior quantidade de jogadores que permaneceram pelo menos cinco anos na NBA, o que deve ser levado em consideração na interpretação de métricas como acurácia, precisão, revocação e F1-Score.

2.2 Normalização dos dados

Como os atributos possuem escalas diferentes, foi aplicada normalização Min-Max nos atributos numéricos. Essa etapa é importante porque tanto o kNN quanto o k-Means dependem de medidas de distância; sem normalização, variáveis com valores naturalmente maiores poderiam dominar o cálculo das distâncias.

A normalização Min-Max transforma cada valor x de uma coluna para a escala $[0, 1]$ por meio da Equação 1.

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}} \quad (1)$$

Na implementação própria, a função de normalização percorre as colunas do *DataFrame*, ignora a coluna `TARGET_5Yrs` e aplica a transformação aos demais atributos. Na comparação com o *scikit-learn*, foi utilizado o `MinMaxScaler`, ajustado sobre os dados de treino e aplicado ao conjunto de teste.

2.3 k-Nearest Neighbors

O kNN é um algoritmo supervisionado baseado em instâncias. Durante o treinamento, o algoritmo apenas armazena os exemplos de treino e seus respectivos rótulos. Para classificar uma nova amostra, calcula-se a distância entre essa amostra e todos os

exemplos de treino, selecionam-se os k vizinhos mais próximos e atribui-se à nova amostra a classe mais frequente entre esses vizinhos.

A implementação utilizou distância Euclidiana, dada por:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (2)$$

em que x e y são dois jogadores representados por seus atributos normalizados, e n é o número de atributos utilizados.

Foram testados os valores $k = \{2, 10, 50, 15\}$. Os valores 2, 10 e 50 foram definidos pelo enunciado, enquanto $k = 15$ foi escolhido como valor adicional intermediário, com o objetivo de avaliar um cenário menos sensível a ruídos que $k = 2$ e menos suavizado que $k = 50$.

2.4 k-Means

O k-Means é um algoritmo não-supervisionado de agrupamento. Seu objetivo é particionar os dados em k grupos, de forma que amostras dentro de um mesmo grupo sejam semelhantes entre si e amostras de grupos diferentes sejam mais distantes.

A implementação seguiu os seguintes passos:

1. Escolher aleatoriamente k amostras como centróides iniciais;
2. Calcular a distância Euclidiana entre cada jogador e cada centróide;
3. Atribuir cada jogador ao centróide mais próximo;
4. Atualizar cada centróide como a média dos jogadores atribuídos ao respectivo cluster;
5. Repetir os passos anteriores até a convergência ou até atingir o número máximo de iterações.

A convergência foi definida pela ausência de deslocamento dos centróides entre duas iterações consecutivas. Nos experimentos, foram testados $k = 2$ e $k = 3$, conforme solicitado. A coluna `TARGET_5Yrs` não foi usada no treinamento do k-Means, sendo utilizada apenas posteriormente para avaliar a relação entre os clusters encontrados e os rótulos reais.

2.5 Métricas de avaliação

Para o kNN, foram calculadas a matriz de confusão, a acurácia, a precisão, a revocação e o F1-Score. Considerando verdadeiro positivo (VP), verdadeiro negativo (VN),

falso positivo (FP) e falso negativo (FN), as métricas são dadas por:

$$\text{Acurácia} = \frac{VP + VN}{VP + VN + FP + FN} \quad (3)$$

$$\text{Precisão} = \frac{VP}{VP + FP} \quad (4)$$

$$\text{Recall} = \frac{VP}{VP + FN} \quad (5)$$

$$\text{F1-Score} = 2 \cdot \frac{\text{Precisão} \cdot \text{Recall}}{\text{Precisão} + \text{Recall}} \quad (6)$$

Para o k-Means, como não há rótulo durante o treinamento, a avaliação foi feita por meio da análise dos centróides e de tabelas cruzadas entre o cluster atribuído e o rótulo real TARGET_5Yrs.

3 Experimentos e Resultados

3.1 Resultados do kNN

A Tabela 1 apresenta os resultados obtidos pela implementação própria do kNN para os quatro valores de k avaliados. A matriz de confusão está organizada com a classe real nas linhas e a classe prevista nas colunas.

Tabela 1: Resultados da implementação própria do kNN.

k	VN	FP	FN	VP	Acurácia	Precisão	Recall	F1-Score
2	69	31	84	84	0,5709	0,7304	0,5000	0,5936
10	64	36	55	113	0,6604	0,7584	0,6726	0,7129
50	60	40	51	117	0,6604	0,7452	0,6964	0,7200
15	55	45	49	119	0,6493	0,7256	0,7083	0,7169

O pior desempenho ocorreu para $k = 2$, com acurácia de 0,5709 e F1-Score de 0,5936. Esse comportamento é esperado, pois valores muito pequenos de k tornam o modelo sensível a ruídos e a casos isolados. Além disso, como $k = 2$ é um número par, há maior possibilidade de empate na votação entre as classes.

Os resultados melhoraram para $k = 10$, $k = 15$ e $k = 50$. O melhor F1-Score da implementação própria foi obtido com $k = 50$, alcançando 0,7200. Esse resultado indica que o uso de uma vizinhança maior suavizou decisões locais muito específicas e permitiu

melhor generalização no conjunto de teste.

3.2 Comparação do kNN com o scikit-learn

A Tabela 2 apresenta os resultados obtidos pelo kNN da biblioteca *scikit-learn*, utilizando distância Euclidiana e os mesmos valores de k .

Tabela 2: Resultados do kNN com scikit-learn.

k	VN	FP	FN	VP	Acurácia	Precisão	Recall	F1-Score
2	72	28	94	74	0,5448	0,7255	0,4405	0,5481
10	56	44	52	116	0,6418	0,7250	0,6905	0,7073
50	54	46	46	122	0,6567	0,7262	0,7262	0,7262
15	53	47	44	124	0,6604	0,7251	0,7381	0,7316

A Figura 1 compara o F1-Score da implementação própria com o obtido pelo *scikit-learn*. Observa-se que os valores são próximos, principalmente para $k = 10$, $k = 15$ e $k = 50$.

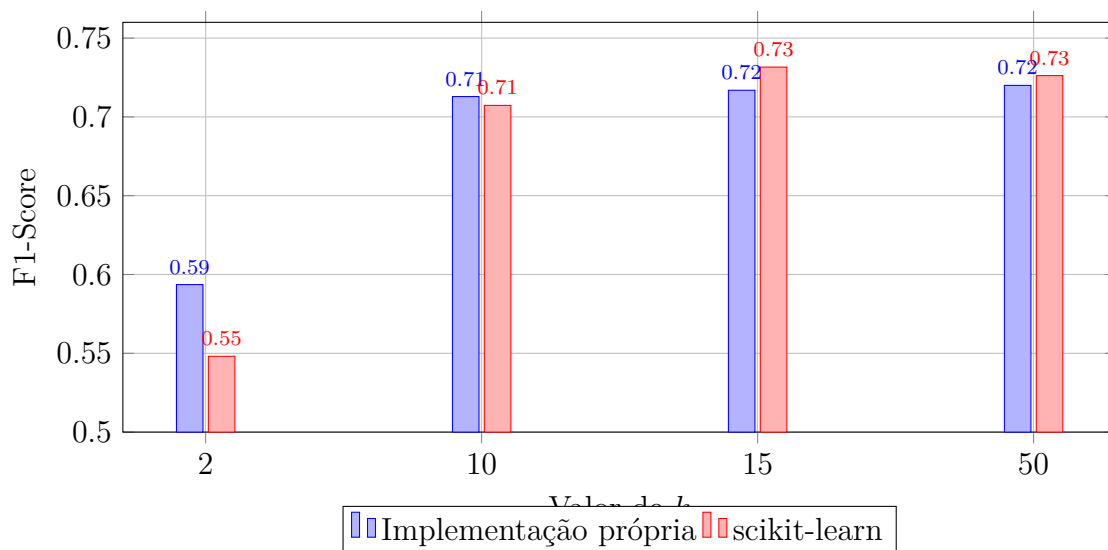


Figura 1: Comparação do F1-Score do kNN entre a implementação própria e o scikit-learn.

As pequenas diferenças entre as duas implementações podem ser explicadas por detalhes de implementação, especialmente na normalização e no critério de desempate. Na implementação própria, a classe vencedora é definida pela contagem de votos dos vizinhos mais próximos. Já a implementação do *scikit-learn* possui rotinas internas otimizadas e um tratamento mais padronizado para ajustes, transformação dos dados e possíveis empates. Apesar disso, a tendência geral dos resultados foi semelhante: valores muito pequenos de

k apresentaram pior desempenho, enquanto valores intermediários ou maiores produziram resultados mais estáveis.

3.3 Resultados do k-Means para $k = 2$

Para $k = 2$, o k-Means separou os jogadores em dois grupos principais. A Tabela 3 apresenta as cinco componentes mais altas de cada centróide normalizado, permitindo interpretar o perfil dominante de cada agrupamento.

Tabela 3: Perfil dos centróides da implementação própria do k-Means para $k = 2$.

Cluster	Atributo	Valor normalizado
0	GP	0,8850
	FT%	0,7049
	MIN	0,6394
	FG%	0,4904
	FGA	0,4786
1	FT%	0,6659
	GP	0,5940
	FG%	0,4360
	MIN	0,2541
	3P%	0,1790

O Cluster 0 apresenta valores altos de jogos disputados e minutos jogados, sugerindo um grupo de jogadores mais utilizados em quadra. O Cluster 1 apresenta menor volume de jogo, principalmente em minutos, indicando um perfil mais próximo de jogadores reservas ou com menor participação.

A Tabela 4 mostra a relação entre os clusters encontrados e a variável real TARGET_5Yrs.

Tabela 4: Relação entre clusters do k-Means e rótulos reais para $k = 2$.

Cluster	TARGET=0	TARGET=1	Proporção TARGET=1
0	69	371	84,3%
1	440	460	51,1%

Com $k = 2$, o algoritmo não separou perfeitamente as duas classes originais, pois o Cluster 1 contém quantidades muito próximas de jogadores que duraram e que não duraram cinco anos. Entretanto, o Cluster 0 possui forte predominância da classe 1, mostrando que

jogadores com maior volume de jogo no ano de novato tendem a permanecer mais tempo na liga.

3.4 Resultados do k-Means para $k = 3$

Para $k = 3$, o algoritmo produziu uma separação mais interpretável do ponto de vista esportivo. A Tabela 5 apresenta as principais componentes dos centróides.

Tabela 5: Perfil dos centróides da implementação própria do k-Means para $k = 3$.

Cluster	Atributo	Valor normalizado
0	FT%	0,6561
	GP	0,5035
	FG%	0,4130
	MIN	0,2004
	3P%	0,1771
1	GP	0,8981
	MIN	0,7770
	FT%	0,7184
	FGA	0,6334
	FGM	0,5853
2	GP	0,8438
	FT%	0,6923
	FG%	0,4913
	MIN	0,4612
	FGA	0,3027

A Tabela 6 apresenta a relação entre os três clusters e os rótulos reais.

Tabela 6: Relação entre clusters do k-Means e rótulos reais para $k = 3$.

Cluster	TARGET=0	TARGET=1	Proporção TARGET=1
0	354	283	44,4%
1	26	158	85,9%
2	129	390	75,1%

A Figura 2 mostra a proporção de jogadores da classe 1 em cada cluster para $k = 2$ e $k = 3$.

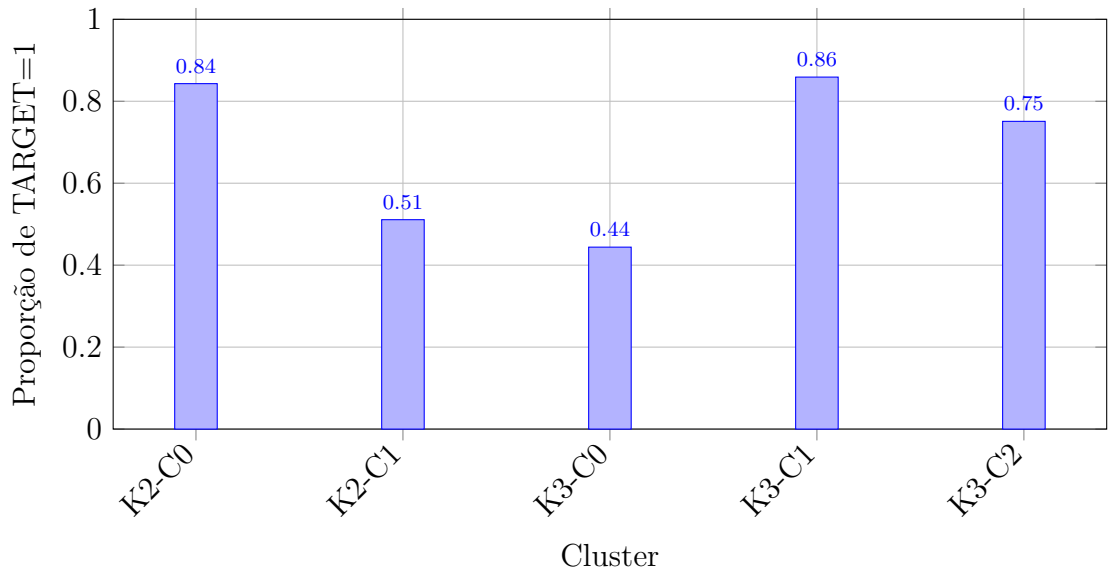


Figura 2: Proporção de jogadores que duraram pelo menos cinco anos em cada cluster.

Com $k = 3$, os agrupamentos podem ser interpretados como três perfis: jogadores de baixa utilização, jogadores de elite ou muito utilizados, e jogadores de rotação. O Cluster 1 possui maior proporção de jogadores com carreira longa, além dos maiores valores de jogos, minutos e tentativas de arremesso, sugerindo um grupo de atletas com papel relevante desde o ano de novato. O Cluster 2 também apresenta forte predominância da classe 1, mas com menor volume ofensivo, indicando jogadores de rotação. O Cluster 0 é o único em que a classe 0 é majoritária, sendo caracterizado por baixa minutagem e menor participação.

3.5 Comparação do k-Means com o scikit-learn

A comparação com o *scikit-learn* foi feita por meio da tabela cruzada entre o cluster atribuído e a variável `TARGET_5Yrs`. A Tabela 7 apresenta os resultados.

Tabela 7: Relação entre clusters do scikit-learn e rótulos reais.

k	Cluster	TARGET=0	TARGET=1	Proporção TARGET=1
2	0	437	462	51,4%
	1	72	369	83,7%
3	0	26	162	86,2%
	1	131	386	74,7%
	2	352	283	44,6%

Os resultados foram muito próximos aos da implementação própria. A principal

diferença é a numeração dos clusters, que não possui significado fixo em aprendizado não-supervisionado. Assim, um grupo identificado como Cluster 0 em uma implementação pode aparecer como Cluster 1 em outra, desde que a composição e o perfil dos pontos sejam equivalentes.

Outra diferença relevante está na inicialização dos centróides. A implementação própria escolhe centróides iniciais aleatoriamente a partir das amostras, enquanto o *scikit-learn* utiliza, por padrão, estratégias mais robustas, como o k-Means++, que tende a espalhar melhor os centróides iniciais e melhorar a convergência. Mesmo assim, as proporções encontradas foram bastante semelhantes, indicando que a implementação própria capturou a mesma estrutura geral dos dados.

4 Discussão

Os resultados do kNN mostram que a escolha do valor de k tem impacto direto na qualidade da classificação. Com $k = 2$, o modelo ficou muito sensível a vizinhos específicos e apresentou baixa revocação, identificando corretamente apenas metade dos jogadores que realmente permaneceram pelo menos cinco anos na liga. Esse comportamento é prejudicial neste problema, pois a classe positiva é justamente a classe de interesse: jogadores com carreira longa.

Para $k = 10$, $k = 15$ e $k = 50$, o desempenho foi mais estável. A acurácia permaneceu próxima de 66%, enquanto o F1-Score ficou em torno de 0,71 a 0,72. O melhor F1-Score da implementação própria foi obtido com $k = 50$, mas a diferença em relação a $k = 15$ foi pequena. Isso indica que, a partir de certo ponto, aumentar o número de vizinhos melhora a estabilidade, mas também pode suavizar demais as fronteiras de decisão.

A comparação com o *scikit-learn* reforça essa interpretação. A biblioteca apresentou melhor F1-Score para $k = 15$, com valor 0,7316, e desempenho muito próximo para $k = 50$. As diferenças observadas não indicam erro na implementação própria; elas são compatíveis com diferenças de normalização, ordenação interna, tratamento de empates e detalhes de implementação.

No k-Means, os resultados indicam que os dados possuem estrutura relacionada ao volume de jogo no ano de novato. Com $k = 2$, o algoritmo conseguiu separar um grupo de jogadores com alta probabilidade de carreira longa, mas não conseguiu distinguir bem todos os jogadores da classe 0. Isso mostra que o problema não é perfeitamente separável apenas com dois grupos.

Com $k = 3$, a estrutura dos dados ficou mais clara. O algoritmo separou jogadores com baixa utilização, jogadores de rotação e jogadores de maior protagonismo. Essa

separação é coerente com o contexto da NBA: jogadores que recebem mais minutos e disputam mais partidas no ano de novato provavelmente demonstraram maior confiança da equipe técnica, melhor desempenho ou maior potencial, fatores associados a carreiras mais longas.

Ainda assim, é importante destacar que a previsão de duração de carreira na NBA é um problema difícil. Estatísticas do ano de novato não capturam todos os fatores relevantes, como lesões, evolução técnica, trocas de equipe, comportamento fora de quadra, oportunidades futuras e contexto tático. Portanto, uma acurácia em torno de 66% não deve ser interpretada como fracasso do modelo, mas como uma limitação natural dos atributos disponíveis.

5 Conclusão

Neste trabalho, foram implementados e avaliados os algoritmos kNN e k-Means aplicados ao conjunto de dados *NBA Rookie Stats*. O kNN foi utilizado para classificar se um jogador teria carreira de pelo menos cinco anos na NBA, enquanto o k-Means foi utilizado para agrupar jogadores sem usar o rótulo real durante o treinamento.

Na implementação própria do kNN, o melhor resultado em F1-Score foi obtido com $k = 50$, com valor 0,7200. O valor $k = 2$ apresentou o pior desempenho, evidenciando a sensibilidade de vizinhanças muito pequenas. Na comparação com o *scikit-learn*, o melhor resultado foi obtido com $k = 15$, com F1-Score de 0,7316. De modo geral, os resultados da implementação própria ficaram próximos aos da biblioteca, indicando que a lógica principal do algoritmo foi implementada corretamente.

Para o k-Means, o algoritmo não conseguiu reproduzir perfeitamente as classes originais, o que é esperado, pois se trata de um método não-supervisionado. No entanto, os clusters encontrados apresentaram forte relação com a permanência dos jogadores na liga. Com $k = 2$, foi identificado um grupo de jogadores com alto volume de jogo e alta proporção de carreiras longas. Com $k = 3$, a separação se tornou mais interpretável, formando grupos compatíveis com jogadores de baixa utilização, jogadores de rotação e jogadores de maior protagonismo.

Assim, conclui-se que os algoritmos implementados foram capazes de extrair padrões relevantes da base de dados. O kNN apresentou desempenho moderado para classificação, limitado pela complexidade do problema e pelos atributos disponíveis, enquanto o k-Means revelou uma estrutura coerente entre volume de jogo no ano de novato e maior probabilidade de permanência na NBA.

Referências

- CHAIMOWICZ, Luiz; NEGRISOLI, Tiago. *Trabalho Prático II: Aprendizado de Máquina*. Universidade Federal de Minas Gerais, Introdução à Inteligência Artificial, 2026.
- PEDREGOSA, Fabian et al. *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, v. 12, p. 2825–2830, 2011.
- HASTIE, Trevor; TIBSHIRANI, Robert; FRIEDMAN, Jerome. *The Elements of Statistical Learning*. 2. ed. New York: Springer, 2009.
- BISHOP, Christopher M. *Pattern Recognition and Machine Learning*. New York: Springer, 2006.